GATE DA 2027

Database Management and Warehousing

ER and relational models, SQL, integrity constraints, normalization, indexing, data preprocessing and data warehouse modelling with the stories of Ramu and Somu

Ramu and Somu once more.
Ramu asks "but why?" about everything. **Somu** answers with a college office: student registers, course lists, a library card catalogue and a shop's monthly sales book.

Syllabus covered (GATE DA, Section 5): ER model; relational model (relational algebra, tuple calculus); SQL; integrity constraints; normal forms; file organization; indexing; data types; data transformation (normalization, discretization, sampling, compression); data warehouse (schema for multidimensional modelling, hierarchies, measures).

Every SQL query, normal-form analysis and numeric example here was run on a real database engine or in code, and the outputs shown are the real outputs.

Contents

1 The Entity–Relationship (ER) Model **3**

1.1 Building blocks . . . 3

1.2 Converting an ER diagram to tables . . . 3

2 The Relational Model and Query Languages **4**

2.1 Keys . . . 4

2.2 Relational algebra . . . 5

2.3 Tuple relational calculus (TRC) and domain calculus . . . 5

3 SQL **6**

3.1 Order of evaluation . . . 7

3.2 Selection, joins and aggregation (all outputs real) . . . 7

3.3 Joins . . . 7

3.4 Subqueries . . . 8

3.5 NULL and three-valued logic . . . 8

3.6 Other statements worth knowing . . . 8

4 Integrity Constraints **9**

5 Functional Dependencies and Normalization **10**

5.1 Functional dependencies and closure . . . 10
5.2 Normal forms . . . 11
5.3 Decomposition: lossless join and dependency preservation . . . 11
5.4 Canonical (minimal) cover . . . 12
5.5 Counting superkeys, and multivalued dependencies . . . 12

6 File Organization and Indexing **13**

6.1 Blocks and file organizations . . . 13
6.2 Indexes . . . 13
6.3 B-trees and B^+-trees . . . 14
6.4 Hashing . . . 14

7 Data Types and Data Transformation (Preprocessing) **15**

7.1 Types of attributes . . . 15
7.2 Missing values and noise . . . 15
7.3 Normalization (rescaling) . . . 15
7.4 Discretization (binning) . . . 16
7.5 Sampling . . . 16
7.6 Compression and data reduction . . . 16

8 Data Warehouse Modelling **17**

8.1 Multidimensional model: facts, dimensions, measures . . . 17
8.2 Schemas . . . 18
8.3 OLAP operations (tested on a real star schema) . . . 18
8.4 The data cube and its lattice . . . 19
8.5 ETL, slowly changing dimensions and speed-ups . . . 19

9 One-page Cheat Sheet **20**

10 Practice Questions (with answers) **20**

1 The Entity–Relationship (ER) Model

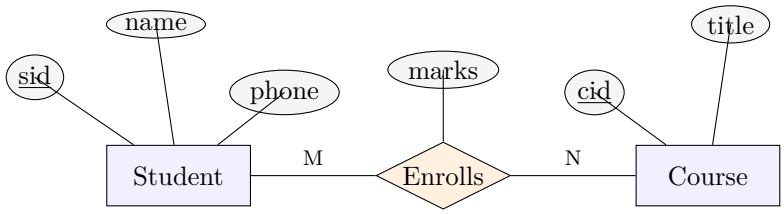
Ramu & Somu: The college office

Somu sits in the college office and looks around. "Everything here is either a *thing* we keep facts about, like a student or a course, or a *link* between things, like 'this student is enrolled in that course'." Ramu: "So a database design starts with a picture of things and links?" "Yes. That picture is the **ER diagram**."

1.1 Building blocks

Vocabulary

- **Entity**: a distinguishable object (a student). An **entity set** is a collection of similar entities (all students); drawn as a rectangle.
- **Attribute**: a property of an entity (name). Drawn as an ellipse. Kinds: *simple* vs *composite* (name = first + last); *single-valued* vs *multivalued* (phone numbers, double ellipse); *stored* vs *derived* (age derived from date of birth, dashed ellipse); a *key attribute* is underlined.
- **Relationship**: an association among entities (enrolls). Drawn as a diamond. The number of entity sets involved is its **degree** (binary, ternary).
- **Cardinality ratio** (how many on each side): 1:1, 1:N, N:1, M:N.
- **Participation**: *total* (every entity must take part; double line) or *partial* (single line).
- **Weak entity**: has no key of its own; it is identified by the key of an *owner* (strong) entity together with its own *partial key* (discriminator). Drawn with a double rectangle and an identifying double diamond. Example: `Dependent` of an `Employee`.
- **Specialization/generalization** ("is-a"): `Student` and `Teacher` are specialisations of `Person`; constraints: *disjoint* vs *overlapping*, *total* vs *partial*.



Here `Student`–`Course` is M:N and the relationship carries its own attribute `marks`: it belongs to the *pair*, not to either entity alone.

1.2 Converting an ER diagram to tables

Conversion rules

- **Strong entity** → one table; its key becomes the primary key. Composite attributes are flattened to their simple parts.
- **Multivalued attribute** → a separate table (owner key, value).
- **Weak entity** → a table containing the owner's key plus its partial key; primary key is the combination.
- **1:N** relationship → no new table: put the key of the 1 side as a foreign key in the N side.
- **1:1** relationship → no new table: add a foreign key to either side (better to the side with

total participation).
- **M:N** relationship → a **new table** holding both keys (together the primary key) and the relationship's attributes.
- **Specialization** → either one table per entity (subclass keyed by the superclass key), or one table for the whole hierarchy with a type column.

Counting shortcut: number of tables = (entity sets) + (M:N relationships) + (multivalued attributes) [for binary relationships].

Counting tables

An ER diagram has entity sets E_1, E_2, E_3. E_1–E_2 is M:N, E_2–E_3 is 1:N, E_1–E_3 is 1:1. Tables: 3 (entities) +1 (the M:N relationship) = **4**; the other two relationships merge into existing tables as foreign keys.

Common GATE traps

(1) A relationship attribute of an M:N relationship cannot be moved into either entity. (2) For a 1:N relationship the foreign key goes on the *many* side. (3) A weak entity has total participation in its identifying relationship. (4) A ternary relationship is not the same as three binary ones.

2 The Relational Model and Query Languages

Ramu & Somu: Registers as tables

Ramu opens a register: rows are students, columns are facts such as name and department. Somu: "A **relation** is exactly such a table. The **schema** is the column headings (written Student($\underline{sid}$, name, dept, cgpa)) and the **instance** is the rows currently in it. Rows are called **tuples**, columns **attributes**. The order of rows and of columns does not matter, and there are no duplicate rows in the pure model."

2.1 Keys

Kinds of keys

- **Superkey**: a set of attributes that uniquely identifies every tuple (any superset of a key).
- **Candidate key**: a *minimal* superkey (remove any attribute and uniqueness is lost).
- **Primary key**: the candidate key chosen for the table; cannot be `NULL`. The others are **alternate keys**.
- **Foreign key**: attributes in one table that refer to the primary key of another (or the same) table. They may repeat and may be `NULL` (unless forbidden).
- **Prime attribute**: part of at least one candidate key; otherwise **non-prime**.

2.2 Relational algebra

Operator	Symbol	Meaning and size
Selection	$\sigma_\theta(R)$	rows satisfying θ; $\le \lvert R\rvert$ rows, same columns
Projection	$\pi_{A_1,\ldots}(R)$	chosen columns, duplicates removed; $\le \lvert R\rvert$ rows
Rename	$\rho_{S(B_1,\ldots)}(R)$	renames relation/attributes
Union, Intersection, Difference	$R \cup S,\ R \cap S,\ R - S$	need *union-compatible* schemas; $\lvert R \cup S\rvert \le \lvert R\rvert + \lvert S\rvert$, $\lvert R - S\rvert \le \lvert R\rvert$
Cartesian product	$R \times S$	every pairing; exactly $\lvert R\rvert \cdot \lvert S\rvert$ rows
Natural join	$R \bowtie S$	product, keep rows equal on the common attributes, drop the duplicate columns
Theta/equi-join	$R \bowtie_\theta S$	product followed by σ_θ
Outer joins	$\bowtie^L$, $\bowtie^R$, $\bowtie^F$ (left, right, full)	keep unmatched rows of left, right, or both sides, padded with `NULL`
Division	$R \div S$	values of R's extra attributes paired with *every* tuple of S

Useful identities and facts

- Join is a derived operator: $R \bowtie_\theta S = \sigma_\theta(R \times S)$. Division is also derived: $R \div S = \pi_X(R) - \pi_X((\pi_X(R) \times S) - R)$.
- Selection pushes below join: $\sigma_\theta(R \bowtie S) = \sigma_\theta(R) \bowtie S$ when θ uses only R's attributes (optimizers do this).
- If R has a foreign key to S's key (non-null), then $\lvert R \bowtie S\rvert = \lvert R\rvert$.
- The **complete set** of operators is $\{\sigma, \pi, \cup, -, \times, \rho\}$: every other operator can be derived.
- `NULL` never equals anything (even `NULL`), so rows with `NULL` join attributes do not match in a join.

Division, and join sizes with NULLs

Let $E(s,c) = \{(a,1),(a,2),(b,1),(c,1),(c,2),(c,3)\}$ (who took which course) and $C(c) = \{1,2\}$. Then $E \div C = \{a,c\}$: only a and c took *both* courses (student b missed course 2).

Let $R(a,b) = \{(1,x),(2,x),(3,y),(4,\text{NULL})\}$ and $S(b,c) = \{(x,10),(x,20),(z,30),(\text{NULL},40)\}$.

- $R \bowtie S$: 4 rows (the two x-tuples of R with the two x-tuples of S).
- Left outer join: $4+2=6$ (adds $(3,y)$ and $(4,\text{NULL})$ with `NULL`s). Full outer join: $6+2=8$.
- $R \times S$: $4 \times 4 = 16$.

2.3 Tuple relational calculus (TRC) and domain calculus

Ramu & Somu: Describe, don't compute

Relational algebra says *how*: "first select, then join, then project". Calculus only says *what* you want: "the set of all t such that t is a student and…". SQL is a blend, but its `SELECT-FROM-WHERE` is closest to this calculus.

Tuple calculus

Form: $\{\, t \mid P(t) \,\}$ where P is a formula built from atoms $R(t)$ (tuple t is in relation R), comparisons such as $t.A > s.B$ or $t.A = 5$, connectives $\wedge, \vee, \neg$, and quantifiers $\exists s \in R$, $\forall s \in R$.

Example, students with a mark above 80: $\{t \mid \text{Student}(t) \wedge \exists e\,(\text{Enroll}(e) \wedge e.sid = t.sid \wedge e.marks > 80)\}$.
Example with $\forall$ (division-like): students who took *every* 4-credit course:

$$\{t \mid \text{Student}(t) \wedge \forall c\,(\text{Course}(c) \wedge c.credits = 4 \Rightarrow \exists e\,(\text{Enroll}(e) \wedge e.sid = t.sid \wedge e.cid = c.cid))\}$$

A formula is **safe** if it can only produce finitely many tuples from values in the database; unsafe examples are like $\{t \mid \neg\text{Student}(t)\}$. Safe tuple calculus is exactly as expressive as relational algebra.
Domain relational calculus uses *variables over attribute values*: $\{\langle n, d\rangle \mid \exists s\,(\langle s, n, d, g\rangle \in \text{Student} \wedge g > 8)\}$ (QBE is based on it).

Common GATE traps

(1) Relational algebra *removes duplicates*; SQL does not unless told to (`DISTINCT`). (2) Natural join joins on *all* common attribute names; with no common attribute it becomes a Cartesian product. (3) Union needs the same number and compatible types of columns. (4) $R \div S$ is not simply "$R - S$".

3 SQL

Ramu & Somu: Asking the office clerk

Ramu wants answers from the student register. Somu: "You do not walk through the files yourself. You write the question in a language the clerk understands and say *what* you want, not *how* to find it. That language is **SQL**." We will use one small college database throughout.

```
CREATE TABLE Student(sid INTEGER PRIMARY KEY, name TEXT NOT NULL,
                     dept TEXT, cgpa REAL CHECK (cgpa BETWEEN 0 AND 10));
CREATE TABLE Course(cid TEXT PRIMARY KEY, title TEXT, credits INTEGER);
CREATE TABLE Enroll(sid INTEGER REFERENCES Student(sid) ON DELETE CASCADE,
                    cid TEXT REFERENCES Course(cid), marks INTEGER,
                    PRIMARY KEY (sid, cid));
```

Student

sid	name	dept	cgpa
1	Asha	CS	8.5
2	Ravi	EE	7.0
3	Meena	CS	9.1
4	Kiran	ME	6.5
5	Tara	EE	NULL

Course

cid	title	credits
C1	DBMS	4
C2	Maths	3
C3	ML	4

Enroll

sid	cid	marks
1	C1	85
1	C2	78
2	C1	62
3	C1	91
3	C3	88
4	C2	55

3.1 Order of evaluation

Logical order in which a SELECT is processed

`FROM` (joins) → `WHERE` (filter rows) → `GROUP BY` → `HAVING` (filter groups) → `SELECT` (compute columns, aggregates) → `DISTINCT` → `ORDER BY` → `LIMIT`.
Hence: `WHERE` cannot use aggregates, `HAVING` can; any non-aggregated column in `SELECT` must appear in `GROUP BY`.

3.2 Selection, joins and aggregation (all outputs real)

```
SELECT s.name, c.title, e.marks
FROM Student s JOIN Enroll e ON s.sid=e.sid JOIN Course c ON c.cid=e.cid
WHERE e.marks > 80 ORDER BY s.name;
```

```
(Asha, DBMS, 85) (Meena, DBMS, 91) (Meena, ML, 88)
```

```
SELECT dept, COUNT(*), COUNT(cgpa), AVG(cgpa) FROM Student GROUP BY dept;
```

```
(CS, 2, 2, 8.8) (EE, 2, 1, 7.0) (ME, 1, 1, 6.5)
```

Note the EE row: `COUNT(*)` = 2 counts both rows, but `COUNT(cgpa)` = 1 and `AVG`= 7.0 because aggregates *ignore NULL*.

```
SELECT sid, AVG(marks) FROM Enroll GROUP BY sid HAVING AVG(marks) > 80;
```

```
(1, 81.5) (3, 89.5)
```

Aggregates and NULL

`COUNT(*)` counts rows; `COUNT(col)`, `SUM`, `AVG`, `MIN`, `MAX` skip `NULL`s. On an empty set `COUNT` gives 0 but `SUM/AVG/MIN/MAX` give `NULL`. `AVG`= `SUM/COUNT(col)` over non-null values only.

3.3 Joins

Join	Result
`INNER JOIN` / `JOIN`	only matching pairs
`LEFT [OUTER] JOIN`	all left rows; unmatched ones get `NULL` on the right
`RIGHT [OUTER] JOIN`	mirror image
`FULL [OUTER] JOIN`	all rows from both sides
`CROSS JOIN` / comma	Cartesian product: $\|R\| \cdot \|S\|$ rows (here $5 \times 3 = 15$)
`NATURAL JOIN`	equality on all common column names

```
SELECT s.name, COUNT(e.cid) FROM Student s LEFT JOIN Enroll e ON s.sid=e.sid GROUP BY s.
    sid;
```

```
(Asha,2) (Ravi,1) (Meena,2) (Kiran,1) (Tara,0) <- Tara appears with 0
```

With an *inner* join the same query drops Tara (she has no enrolment). Counting `e.cid` (not `*`) gives 0 for her; `COUNT(*)` would wrongly give 1.

3.4 Subqueries

Ramu & Somu: A question inside a question

"Who scored above the class average?" needs the average first. Somu writes it as a **subquery**: a query in brackets used as a value, a list, or a yes/no test. A **correlated** subquery refers to the outer row and is re-evaluated for each outer row.

```
-- students above the average cgpa (AVG ignores NULL: (8.5+7+9.1+6.5)/4 = 7.775)
SELECT name FROM Student WHERE cgpa > (SELECT AVG(cgpa) FROM Student); -- Asha, Meena

-- students enrolled in no course (correlated NOT EXISTS)
SELECT name FROM Student s WHERE NOT EXISTS (SELECT 1 FROM Enroll e WHERE e.sid=s.sid);
    -- Tara

-- second highest cgpa
SELECT MAX(cgpa) FROM Student WHERE cgpa < (SELECT MAX(cgpa) FROM Student); -- 8.5

-- division in SQL: students who took EVERY 4-credit course (double NOT EXISTS)
SELECT name FROM Student s WHERE NOT EXISTS (
  SELECT * FROM Course c WHERE c.credits=4 AND NOT EXISTS (
    SELECT * FROM Enroll e WHERE e.sid=s.sid AND e.cid=c.cid)); -- Meena
```

Set operations on the C1 and C2 enrolments: `INTERSECT` gives $\{1\}$, `UNION` gives $\{1,2,3,4\}$, `EXCEPT` (C1 minus C2) gives $\{2,3\}$. Set operators remove duplicates (`UNION ALL` keeps them).

3.5 NULL and three-valued logic

NULL rules

Any comparison with `NULL` yields **UNKNOWN**, not true or false. A `WHERE` keeps only rows where the condition is **TRUE**. Test with `IS NULL`. Truth tables: $\mathrm{T} \wedge \mathrm{U} = \mathrm{U}$, $\mathrm{F} \wedge \mathrm{U} = \mathrm{F}$, $\mathrm{T} \vee \mathrm{U} = \mathrm{T}$, $\mathrm{F} \vee \mathrm{U} = \mathrm{U}$, $\neg\mathrm{U} = \mathrm{U}$.

```
SELECT COUNT(*) FROM Student WHERE cgpa = NULL; -- 0 (never true!)
SELECT name FROM Student WHERE cgpa <> 8.5; -- Ravi, Meena, Kiran (Tara lost)
SELECT name FROM Student WHERE cgpa <> 8.5 OR cgpa IS NULL; -- Ravi, Meena, Kiran, Tara
```

Common GATE traps

(1) `x NOT IN (subquery)` returns *nothing at all* if the subquery contains a `NULL`: with $A = \{1,2,3\}$ and $B = \{1, \mathrm{NULL}\}$, `SELECT x FROM A WHERE x NOT IN (SELECT y FROM B)` returns an empty result, while the `NOT EXISTS` version returns $\{2,3\}$. (2) `WHERE` cannot contain `COUNT(*)`; use `HAVING`. (3) `COUNT(*)` versus `COUNT(col)` with NULLs. (4) `ORDER BY` sorts `NULL`s first or last depending on the system. (5) `LIKE '_a%'`: `_` is exactly one character and `%` is any string; here it matches Ravi and Tara. (6) `DISTINCT` applies to the whole selected row, not one column.

3.6 Other statements worth knowing

```
INSERT INTO Course VALUES ('C4','AI',3);
UPDATE Student SET cgpa = cgpa + 0.1 WHERE dept = 'CS';
DELETE FROM Enroll WHERE marks < 40;
ALTER TABLE Student ADD COLUMN email TEXT;
DROP TABLE Enroll; -- removes structure and data
```

```
CREATE VIEW TopStudents AS SELECT * FROM Student WHERE cgpa > 8; -- a stored query
CREATE INDEX idx_dept ON Student(dept); -- speeds up lookups
```

A **view** is a virtual table (its definition is stored, not its data). `DELETE` removes rows (can be rolled back and filtered), `TRUNCATE` empties a table quickly, `DROP` removes the table itself.

4 Integrity Constraints

Ramu & Somu: The strict librarian

Somu's librarian refuses to issue a card without a name, refuses two cards with the same number, and refuses to record a loan for a book that is not in the catalogue. "These rules are **integrity constraints**: the database itself rejects wrong data, no matter who types it."

Types of constraints

- **Domain constraint**: values must come from the declared type and satisfy `CHECK` (e.g. $0 \le \text{cgpa} \le 10$); `NOT NULL`.
- **Key constraint**: `UNIQUE`/`PRIMARY KEY` values cannot repeat.
- **Entity integrity**: no component of a primary key can be `NULL`.
- **Referential integrity**: every non-null foreign-key value must exist as a primary-key (or unique) value in the referenced table.
- **Semantic constraints**: business rules through triggers or assertions.

Referential actions when the referenced row is deleted or updated

`NO ACTION/RESTRICT`: reject the operation. `CASCADE`: delete/update the referencing rows too. `SET NULL`: set the foreign key to `NULL`. `SET DEFAULT`: set it to the default.
Which operations can violate referential integrity? **Insert** into the *referencing* (child) table, **delete** from the *referenced* (parent) table, and **update** of either key.

```
INSERT INTO Enroll VALUES (9,'C1',50); -- student 9 does not exist
INSERT INTO Student VALUES (6,'Zed','CS',11); -- violates CHECK (cgpa BETWEEN 0 AND 10)
INSERT INTO Student VALUES (1,'Dup','CS',5); -- duplicate primary key
DELETE FROM Course WHERE cid='C1'; -- still referenced by Enroll
DELETE FROM Student WHERE sid=1; -- ON DELETE CASCADE removes Asha's Enroll rows
```

```
FOREIGN KEY constraint failed
CHECK constraint failed: cgpa BETWEEN 0 AND 10
UNIQUE constraint failed: Student.sid
FOREIGN KEY constraint failed
(the delete succeeds; the SELECT afterwards finds no Enroll rows for sid 1)
```

Common GATE traps

(1) A foreign key may be `NULL` ("no reference yet"); a primary key may not. (2) `UNIQUE` allows `NULL`s (how many depends on the system); `PRIMARY KEY` does not. (3) Referential integrity only checks that the *value exists*; it says nothing about which row is "correct". (4) Deleting a parent row is the dangerous operation, not inserting one.

5 Functional Dependencies and Normalization

Ramu & Somu: The messy register

Somu keeps one big register with columns `student, course, teacher, teacher's room`. "Look at the trouble," he says. If a teacher changes rooms, *every* row mentioning her must be edited (an **update anomaly**). A new teacher with no students yet cannot be recorded (an **insertion anomaly**). Deleting the last student of a course also deletes the only record of the teacher (a **deletion anomaly**). "The cure is to split the register so that each fact is stored *once*: that is **normalization**."

5.1 Functional dependencies and closure

Functional dependency

$X \to Y$ ("X determines Y") means: any two tuples that agree on X also agree on Y. It is a statement about *all legal instances*, a rule of the real world, not something you read off one table. X is a **superkey** of R iff $X \to R$.
Trivial if $Y \subseteq X$. **Armstrong's axioms:** *reflexivity* ($Y \subseteq X \Rightarrow X \to Y$), *augmentation* ($X \to Y \Rightarrow XZ \to YZ$), *transitivity* ($X \to Y, Y \to Z \Rightarrow X \to Z$). They are sound and complete. Derived: *union* ($X \to Y, X \to Z \Rightarrow X \to YZ$), *decomposition* ($X \to YZ \Rightarrow X \to Y$ and $X \to Z$), *pseudo-transitivity* ($X \to Y$, $WY \to Z \Rightarrow WX \to Z$).

Attribute closure X^+ (the workhorse)

Start with $X^+ = X$. Repeat: if some FD $L \to R$ has $L \subseteq X^+$, add R to X^+. Stop when nothing changes. Then $X \to Y$ follows from F iff $Y \subseteq X^+$, and X is a superkey iff X^+ contains all attributes.
Candidate-key shortcuts: attributes that appear *only* on the left of every FD (or on no FD) must be in every key; attributes that appear only on the right are in no key. Start from the compulsory attributes, compute the closure, and add attributes only if needed.

Finding keys: $R(A, B, C, D, E)$ with $F = \{A \to B,\ BC \to E,\ ED \to A\}$

C and D never appear on a right side, so they are in every key. Try CD: $(CD)^+ = CD$, not everything. Add one more attribute:

- $(ACD)^+$: $A \to B$ gives $ACDB$; $BC \to E$ gives E. All of $ABCDE$. ✓
- $(BCD)^+$: $BC \to E$ gives E; $ED \to A$ gives A. All of $ABCDE$. ✓
- $(CDE)^+$: $ED \to A$ gives A; $A \to B$ gives B. All of $ABCDE$. ✓

So the candidate keys are $\{ACD,\ BCD,\ CDE\}$. Every attribute is prime. (Verified by brute force.)

5.2 Normal forms

1NF, 2NF, 3NF, BCNF

- **1NF**: every attribute is atomic (no repeating groups or sets in a cell).
- **2NF**: 1NF and no *partial dependency*: no non-prime attribute depends on a *proper subset* of a candidate key. (Only matters for composite keys.)
- **3NF**: for every non-trivial FD $X \to A$: X is a superkey **or** A is prime. (Equivalently: no non-prime attribute is transitively dependent on a key.)
- **BCNF**: for every non-trivial FD $X \to Y$: X is a superkey. (Stricter: the "or A is prime" escape is removed.)

$$\text{BCNF} \subset \text{3NF} \subset \text{2NF} \subset \text{1NF}.$$

The mnemonic: *every non-key attribute must depend on "the key (1NF), the whole key (2NF), and nothing but the key (3NF/BCNF), so help me Codd."*

Identifying the highest normal form, five relations (all checked by program)

Relation and FDs	Candidate keys	Highest form and why
$R(A,B,C,D)$; $A \to B,\ C \to D$	AC	**1NF**: B depends on part A of the key (partial)
$R(A,B,C)$; $A \to B,\ B \to C$	A	**2NF**: C depends on A through B (transitive), B not a key
$R(S,C,I)$; $SC \to I,\ I \to C$	$SC,\ SI$	**3NF, not BCNF**: $I \to C$ but I is not a superkey; C is prime
$R(A,B,C)$; $AB \to C,\ C \to B$	$AB,\ AC$	**3NF, not BCNF**: $C \to B$, C not a superkey, B prime
$R(A,B,C,D)$; $A \to B, B \to C, C \to D, D \to A$	A, B, C, D	**BCNF**: every determinant is a key

Fact: every relation with only two attributes is in BCNF.

5.3 Decomposition: lossless join and dependency preservation

Ramu & Somu: Splitting without losing information

When Somu splits a table he wants two things. First, he must be able to rebuild the original by joining the pieces, *no more and no fewer* rows (**lossless**). Second, the rules (F) should still be checkable on the pieces without joining them (**dependency preserving**).

Tests

- **Lossless (binary):** R splits into R_1, R_2 losslessly iff $R_1 \cap R_2 \to R_1$ or $R_1 \cap R_2 \to R_2$, i.e. the common attributes form a key of one piece. For more than two pieces use the **chase** (tableau) test.
- **Dependency preserving:** the union of FDs that hold within each piece (projections of F) implies all of F.
- **BCNF** decomposition is always lossless but may *lose* dependencies. **3NF** synthesis (from a canonical cover) is *always* lossless and dependency preserving. Neither normal form removes *all* redundancy caused by multivalued dependencies.

Checking decompositions of $R(A, B, C)$ (verified by chase)

- With $A \to B$: $\{AB, AC\}$ is **lossless** (common attribute A is a key of AB). $\{AB, BC\}$ is **lossy** (common attribute B is a key of neither).
- With $B \to C$: $\{AB, BC\}$ is lossless (B is a key of BC).
- $R(S, C, I)$ with $SC \to I,\ I \to C$: BCNF decomposition gives $\{SI,\ IC\}$ (lossless since I is the key of IC) but the dependency $SC \to I$ can no longer be checked inside one table: it is **not preserved**.

5.4 Canonical (minimal) cover

Computing a canonical cover F_c

(1) Split right sides so each FD has a single attribute on the right. (2) Remove **extraneous attributes** from left sides: in $XA \to B$, A is extraneous if $B \in X^+$ computed with F. (3) Remove **redundant FDs**: delete $X \to A$ if $A \in X^+$ computed from the remaining FDs. The result is not unique, but all minimal covers have the same closure.

$F = \{A \to BC,\ B \to C,\ A \to B,\ AB \to C\}$

Split: $A \to B,\ A \to C,\ B \to C,\ A \to B,\ AB \to C$. Remove the duplicate $A \to B$. In $AB \to C$, B alone already gives C (via $B \to C$), so it is redundant. $A \to C$ is redundant because $A \to B \to C$. The canonical cover is $\{A \to B,\ B \to C\}$ (verified by program).

5.5 Counting superkeys, and multivalued dependencies

- If a relation of n attributes has a *single* candidate key of k attributes, it has 2^{n-k} superkeys. For $R(A, B, C, D)$ with key A: $2^3 = 8$. With two single-attribute keys A and B on four attributes: $8 + 8 - 4 = 12$ superkeys (inclusion–exclusion).
- **Multivalued dependency** $X \twoheadrightarrow Y$: for each X-value, the set of Y-values is independent of the remaining attributes. A relation is in **4NF** if every non-trivial MVD $X \twoheadrightarrow Y$ has X as a superkey. Classic example: `(person, phone, hobby)` where phones and hobbies are independent: storing all combinations is redundant; split into `(person, phone)` and `(person, hobby)`.

Common GATE traps

(1) Closure is computed with *all* FDs, and $X \to Y$ is judged by $Y \subseteq X^+$. (2) 3NF allows $X \to A$ with A prime even if X is not a key; BCNF does not. (3) 2NF violations need a *proper subset of a candidate key* on the left. (4) Check *all* candidate keys when deciding which attributes are prime. (5) BCNF decomposition can lose FDs; 3NF cannot. (6) Lossless does *not* imply dependency preserving, and vice versa. (7) A table with a single-attribute candidate key and no other FDs is automatically 2NF (no partial dependencies are possible).

6 File Organization and Indexing

Ramu & Somu: Finding a book in the library

Ramu wants one book. Plan A: scan every shelf (a **heap file**: records in arrival order). Plan B: shelves sorted by title, so he can open the middle and halve the search (an **ordered file**). Plan C: a formula that tells him the exact shelf from the title (a **hashed file**). Plan D: a small **card catalogue** listing the title and shelf number, itself sorted: that is an **index**. Everything here is measured in *block (disk page) accesses*, because reading a disk block costs far more than anything the CPU does in memory.

6.1 Blocks and file organizations

Basic calculations

Block size B, record size R: **blocking factor** $bfr = \lfloor B/R \rfloor$ records per block (unspanned records). Number of blocks for r records: $b = \lceil r/bfr \rceil$.

Organization	Search on key	Insert	Range query
Heap (unordered)	$b/2$ average, b worst	1 (append)	b
Sorted (ordered)	$\lceil \log_2 b \rceil$	expensive (shift records)	good
Hashed	about 1	about 1	poor

6.2 Indexes

Kinds of index

- **Primary index**: on the *ordering key* field of a sorted file. **Clustering index**: on an ordering *non-key* field. **Secondary index**: on a non-ordering field; there can be many, and it must be **dense**.
- **Dense** index: one entry for *every* record. **Sparse** index: one entry per *block* (only possible if the data file is sorted on that field).
- A file can have at most **one** primary or clustering index (it can only be sorted one way) but many secondary indexes.
- **Multilevel index**: build an index on the index until the top level fits in one block. Search cost = number of levels +1 data block.

Worked numbers: 30 000 records of 100 bytes, block 1024 bytes, index entry = 8 B key + 6 B pointer

- $bfr = \lfloor 1024/100 \rfloor = 10$, so $b = 30\,000/10 = 3000$ blocks. (24 bytes per block are wasted.)
- Binary search on the data file: $\lceil \log_2 3000 \rceil =$ **12** block accesses (since $2^{11} = 2048 < 3000 \leq 4096$).
- Index entries per block: $\lfloor 1024/14 \rfloor = 73$.
- **Sparse primary index**: one entry per data block, 3000 entries, so $\lceil 3000/73 \rceil = 42$ index blocks; binary search in the index $\lceil \log_2 42 \rceil = 6$, plus 1 data block: **7** accesses.
- **Dense secondary index**: 30 000 entries, $\lceil 30\,000/73 \rceil = 411$ index blocks; $\lceil \log_2 411 \rceil = 9$, plus 1: **10** accesses.
- **Multilevel** on the dense index: level sizes $411 \to \lceil 411/73 \rceil = 6 \to 1$ block, so 3 index levels +1 data block = **4** accesses.

6.3 B-trees and B$^+$-trees

Ramu & Somu: A tree that stays short and fat

A binary tree of a million keys is 20 levels deep: 20 disk reads. Somu widens each node to fill a whole disk block, holding about 70 keys. Now the same million keys need only 3 or 4 levels. "A **B$^+$-tree** is a balanced, *wide* search tree designed for disks: all leaves are at the same depth, nodes are at least half full, and inserts split nodes instead of making the tree lopsided."

B$^+$-tree facts

- **Internal nodes** hold only keys and child pointers (a router). **All data pointers are in the leaves**, and leaves are linked in a list, so range scans follow the chain. In a B-tree, data pointers can also sit in internal nodes.
- **Order** p (internal node): up to p child pointers and $p-1$ keys. Choose the largest p with $p \cdot P_{ptr} + (p-1) \cdot K \le B$. Every non-root internal node has at least $\lceil p/2 \rceil$ children; the root has at least 2.
- Leaf of capacity p_{leaf}: largest p_{leaf} with $p_{leaf}(K + P_{rec}) + P_{ptr} \le B$ (one extra pointer to the next leaf). Non-root leaves are at least $\lceil p_{leaf}/2 \rceil$ full.
- Search, insert, delete all cost $O(\log_p N)$ block accesses. Height grows only when the root splits.

Order calculation: $B = 1024$, key $K = 8$, block pointer 6, record pointer 7

Internal: $6p + 8(p-1) \le 1024 \Rightarrow 14p \le 1032 \Rightarrow p = \mathbf{73}$ (check: $73 \cdot 6 + 72 \cdot 8 = 1014 \le 1024$; $p = 74$ gives 1028). Leaf: $15\,p_{leaf} + 6 \le 1024 \Rightarrow p_{leaf} = \mathbf{67}$. Minimum fill: 37 children internally, 34 entries in a leaf.
Maximum records indexed by a tree with k levels (root = level 1, leaves at level k): $73^{k-1} \cdot 67$.

levels k	1	2	3	4	5
max records	67	4 891	357 043	26 064 139	1 902 682 147

So 1 million records need at most 4 levels, i.e. 4 block reads to find any record (plus one for the data block).

6.4 Hashing

- **Static hashing:** $h(K)$ mod M chooses one of M buckets (blocks). Collisions beyond one block go to **overflow chains**. Equality lookups ≈ 1 block access; range queries are not supported.
- **Dynamic hashing** (*extendible* and *linear* hashing) lets the file grow without rehashing everything: extendible hashing keeps a **directory** indexed by the first *global depth* bits of the hash, each bucket has a *local depth*, and a full bucket splits (the directory doubles only when local depth would exceed global depth).
- Rule of thumb: **B$^+$-tree** for range and equality; **hash** for equality only; bitmap indexes for low-cardinality columns in warehouses.

Common GATE traps

(1) A sparse index needs a *sorted* data file; a secondary index must be dense. (2) Compute bfr with a floor and b with a ceiling. (3) The number of index entries at the next level equals the number of *blocks* at the level below. (4) In a B$^+$-tree the leaf order differs from the internal order because leaves store record pointers. (5) A binary search over b blocks costs $\lceil \log_2 b \rceil$ block

accesses. (6) An index speeds up reads but slows down inserts and deletes (the index must be updated too).

7 Data Types and Data Transformation (Preprocessing)

Ramu & Somu: Cleaning the exam sheets

Before any analysis Somu collects the marks sheets from five colleges. One marks out of 100, another out of 10, some boxes are empty, a few say "1000". "If we feed this straight into a program the answers will be nonsense," he says. "First we *clean* and *transform* the data: put everything on one scale, fill or drop the gaps, shrink the size." This is **data preprocessing**.

7.1 Types of attributes

Type	What is meaningful	Examples
Nominal	equality only (labels)	colour, department, PIN code
Ordinal	order, but not differences	grades A>B>C, ratings 1–5
Interval	order and differences; no true zero	temperature in °C, calendar year
Ratio	order, differences, *ratios*; true zero	height, salary, age, count

Also: **discrete** (countable) vs **continuous**; **binary** attributes (symmetric or asymmetric). Only ratio data allow statements like "twice as much" (40°C is *not* twice as hot as 20°C).

7.2 Missing values and noise

Handling missing values: ignore the record, fill with a constant, the attribute **mean/median** (or the class-wise mean), the most likely value (prediction), or flag it with an indicator column. **Noise/outliers:** smoothing by binning (below), regression, or clustering; a common outlier rule flags values outside $[Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}]$.

7.3 Normalization (rescaling)

Ramu & Somu: Making rupees and kilometres comparable

A distance method treats "salary = 50 000" and "age = 30" by raw size, so salary would swamp age. Somu rescales each attribute so no single one dominates.

Data used below (12 prices, verified): $d = \{4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34\}$, with min $= 4$, max $= 34$, mean $\mu = 20.33$, median 22.5, population standard deviation $\sigma = 8.966$.

Normalization methods

- **Min–max** to $[0, 1]$: $v' = \dfrac{v - \min}{\max - \min}$. To a new range $[a, b]$: $v' = a + \dfrac{(v - \min)(b - a)}{\max - \min}$. Preserves the shape, but is sensitive to outliers. For $v = 21$: $\frac{21-4}{30} = \mathbf{0.5667}$; for $v = 8$: 0.1333.

- **Z-score**: $v' = \dfrac{v - \mu}{\sigma}$ (mean 0, standard deviation 1). For $v = 21$: $\frac{21-20.33}{8.966} = \mathbf{0.0744}$; $v = 4$: -1.8217; $v = 34$: 1.5243. (With the sample standard deviation 9.3647 it would be 0.0712: read whether the question says population or sample.)
- **Decimal scaling**: $v' = \dfrac{v}{10^j}$ with j the smallest integer such that $\max|v'| < 1$. Here max = 34, so $j = 2$: $4 \to 0.04$, $15 \to 0.15$.

7.4 Discretization (binning)

Equal-width and equal-depth bins

Equal-width (equal interval): width $w = \frac{\max - \min}{k}$. For $k = 3$: $w = 10$, bins $[4, 14)$, $[14, 24)$, $[24, 34]$:

$$\{4, 8, 9\},\ \{15, 21, 21\},\ \{24, 25, 26, 28, 29, 34\} \quad \text{(sizes } 3, 3, 6 : \text{ uneven)}.$$

Equal-depth (equal frequency): each bin has the same number of values, here 4:

$$\{4, 8, 9, 15\},\ \{21, 21, 24, 25\},\ \{26, 28, 29, 34\}.$$

Smoothing by bin means: replace by the bin mean: 9, 22.75, 29.25 (each value in the bin becomes that number).
Smoothing by bin boundaries: replace each value by the nearer of the bin's min and max: $\{4, 4, 4, 15\},\ \{21, 21, 25, 25\},\ \{26, 26, 26, 34\}$.
Smoothing by bin medians: 8.5, 22.5, 28.5.

Other discretization: concept hierarchies (age → young/middle/old), clustering, entropy-based (supervised) splitting.

7.5 Sampling

- **Simple random sample without replacement** (SRSWOR): each record has probability $\frac{n}{N}$ of being chosen. With $N = 10$, $n = 3$: 0.3.
- **With replacement** (SRSWR): a record can appear twice. Probability that a specific record appears at least once in $n = 3$ draws from 10: $1 - 0.9^3 = 0.271$.
- **Stratified sampling**: divide the data into groups (strata, e.g. by class) and sample from each, so rare groups are represented.
- **Systematic sampling**: every k-th record after a random start. **Cluster sampling**: pick whole groups at random.
- **Reservoir sampling**: choose n records uniformly from a stream of unknown length in one pass.

7.6 Compression and data reduction

Compression

Lossless compression recovers the data exactly (run-length encoding, Huffman coding, dictionary methods such as LZW); **lossy** compression does not (JPEG, wavelet transforms, PCA-style projection). Data reduction also includes **dimensionality reduction** (PCA, feature selection), **numerosity reduction** (sampling, histograms, clustering) and **aggregation** (data cubes).

- **Run-length encoding**: `WWWWBBBWW` → `4W3B2W` (9 symbols to 6). It helps only when there

are long runs.

- **Huffman coding**: frequent symbols get short codes; no code is a prefix of another. Frequencies a:45, b:13, c:12, d:16, e:9, f:5 (per 100 symbols) give code lengths $a = 1$, $b = c = d = 3$, $e = f = 4$ and total cost $45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4 = \mathbf{224}$ bits, versus **300** bits for a fixed 3-bit code. Construction: repeatedly merge the two smallest weights (here $5 + 9 = 14$, $12 + 13 = 25$, $14 + 16 = 30$, $25 + 30 = 55$, $45 + 55 = 100$).

Common GATE traps

(1) Min–max is *not* the same as z-score; z-scores can be negative and exceed 1. (2) Equal-width bins can be very unevenly filled; equal-depth bins hold equal counts but have unequal widths. (3) Check the interval endpoints: $[4, 14)$ includes 4 but not 14. (4) Fit the scaling parameters on training data, then apply them unchanged to test data. (5) Lossy compression is irreversible. (6) Temperature in °C is interval data, not ratio.

8 Data Warehouse Modelling

Ramu & Somu: The shop's yearly review

Ramu runs a chain of shops. The cashier's database (**OLTP**) records each sale quickly and correctly: "one pen sold at 10:42". At year-end the owner asks: "Total sales by region, by quarter, by product category, compared with last year." Running such a huge question on the live billing system would slow the counters. Somu says: "Copy the history into a separate store built for *questions*, not for transactions. That is a **data warehouse**."

Definition (Inmon)

A data warehouse is **subject-oriented** (organised around subjects such as sales, not applications), **integrated** (data from many sources made consistent), **time-variant** (keeps history), and **non-volatile** (data is loaded and read, not updated in place).

	OLTP (operational)	**OLAP (warehouse)**
Purpose	day-to-day transactions	analysis, decision support
Data	current, detailed, normalised	historical, summarised, denormalised
Operations	many short inserts/updates	complex read-mostly queries
Users	clerks, many	analysts, few
Size	MB–GB	GB–TB+
Design	ER/normalised	star/snowflake

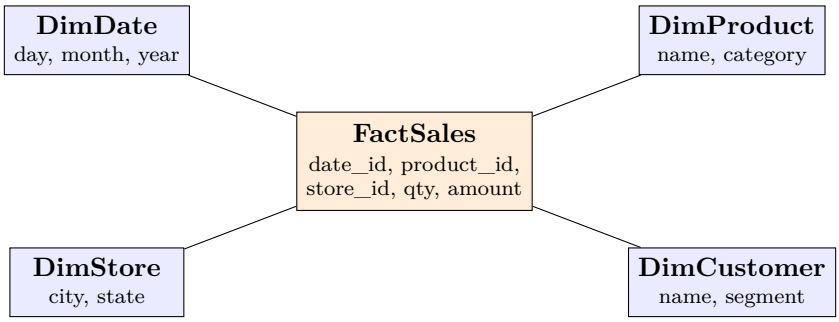
8.1 Multidimensional model: facts, dimensions, measures

Concepts

- **Fact table**: one row per business event at a chosen **granularity** (e.g. one product sold on one day in one shop). Columns: foreign keys to every dimension plus numeric **measures** (quantity, amount). It is the biggest table.
- **Dimension table**: descriptive context (Date, Product, Store, Customer). Wide, relatively

small, denormalised; keyed by a **surrogate key**.
- **Hierarchy** within a dimension: Date: day → month → quarter → year; Location: street → city → state → country. Hierarchies make *roll-up* and *drill-down* possible.
- **Measure types** (by how they aggregate): **distributive** (can be computed from partial aggregates of partitions: `SUM, COUNT, MIN, MAX`); **algebraic** (computed from a fixed number of distributive ones: `AVG`= `SUM`/`COUNT`, standard deviation); **holistic** (no constant-size partial summary: `MEDIAN, MODE, RANK`).
- **Additive** measures (amount) can be summed across all dimensions; **semi-additive** (account balance) cannot be summed across time; **non-additive** (ratios, percentages) cannot be summed at all.

8.2 Schemas



Star, snowflake and fact constellation

- **Star schema** (above): one central fact table directly joined to *denormalised* dimension tables. Fewest joins, fastest queries, some redundancy in dimensions.
- **Snowflake schema**: the dimensions are *normalised* into sub-tables (DimProduct → DimCategory). Less redundancy and storage, but more joins and slower queries.
- **Fact constellation (galaxy)**: several fact tables (Sales, Shipping) sharing dimension tables.
- A **data mart** is a department-level subset of a warehouse (single subject).

8.3 OLAP operations (tested on a real star schema)

Ramu & Somu: Turning the cube

Somu pictures sales as a cube whose three edges are *time*, *product* and *place*. Each small cell holds a total. Looking at the cube from different sides is OLAP.

We loaded a tiny warehouse: DimDate (4 rows: Jan/Feb of 2025 and 2026), DimProduct (3: Pen, Book (Stationery), Mug (Kitchen)), FactSales (7 rows; total amount 1080).

Operation	Meaning	Example result
Roll-up	climb a hierarchy or drop a dimension (less detail)	sales by month → by year: 2025: 410, 2026: 670
Drill-down	the reverse: more detail	year → (year, category): 2025 Kitchen 60, Stationery 350; 2026 Kitchen 120, Stationery 550
Slice	fix *one* dimension to one value	year = 2025: Kitchen 60, Stationery 350
Dice	select values on *two or more* dimensions (a sub-cube)	year = 2026 and category in {Kitchen}
Pivot (rotate)	swap rows and columns of the view	categories as columns instead of rows

```sql
-- roll-up: total sales per year
SELECT year, SUM(amount) FROM FactSales f JOIN DimDate d ON f.d=d.d GROUP BY year;
-- drill-down: add category
SELECT year, category, SUM(amount)
FROM FactSales f JOIN DimDate d ON f.d=d.d JOIN DimProduct p ON f.p=p.p
GROUP BY year, category;
-- slice: year = 2025
... WHERE year = 2025 GROUP BY category;
```

```
(2025, 410) (2026, 670)
(2025, Kitchen, 60) (2025, Stationery, 350) (2026, Kitchen, 120) (2026, Stationery, 550)
(Kitchen, 60) (Stationery, 350)
```

8.4 The data cube and its lattice

Counting cuboids

For n dimensions *without hierarchies*, the **cube** (CUBE operator) contains $\mathbf{2^n}$ **cuboids**: one for every subset of dimensions to group by. The n-dimensional cuboid is the **base cuboid**; the 0-dimensional one is the **apex** (grand total). Examples: $n = 3 \to 8$, $n = 4 \to 16$, $n = 10 \to 1024$. With hierarchies, if dimension i has L_i levels (not counting the virtual level "all"), the number of cuboids is $\prod_{i=1}^{n}(L_i + 1)$. Time (day, month, year) $\to 4$, location (city, state, country) $\to 4$, product (item, category) $\to 3$ gives $4 \cdot 4 \cdot 3 = \mathbf{48}$.
Maximum number of cells in the base cuboid $= \prod |\text{dimension}_i|$: dimensions of sizes $10, 5, 4$ give 200 cells. Real cubes are *sparse*: most cells are empty.

8.5 ETL, slowly changing dimensions and speed-ups

- **ETL**: **E**xtract from sources, **T**ransform (clean, convert types, rescale, deduplicate, apply business rules), **L**oad into the warehouse. The preprocessing of the previous section lives in the transform step.
- **Slowly changing dimensions** (a customer moves city): **Type 1** overwrite (history lost); **Type 2** add a new row with a new surrogate key and validity dates (full history); **Type 3** add a column for the previous value (limited history).
- **Materialised views** and pre-computed cuboids trade storage for speed; **bitmap indexes** (one bit-vector per distinct value) are ideal for low-cardinality columns such as gender or category, because `AND/OR` of bitmaps is fast.
- Warehouse approaches: **ROLAP** (relational tables), **MOLAP** (array-based multidimensional storage), **HOLAP** (hybrid).

Common GATE traps

(1) Snowflake normalises the *dimensions*, not the fact table. (2) Slice fixes one dimension; dice restricts several. (3) Roll-up goes *up* the hierarchy (fewer details) *or* removes a dimension. (4) The number of cuboids is 2^n only without hierarchies. (5) Median is *holistic*; average is *algebraic*, not distributive. (6) A fact table row must be at one consistent granularity. (7) Warehouse data are mostly loaded and read, not updated.

9 One-page Cheat Sheet

ER, relational model, algebra

Tables = entity sets + M:N relationships + multivalued attributes. 1:N: foreign key on the N side. Weak entity key = owner key + partial key. $|R \times S| = |R||S|$; $R \bowtie S$ with an FK to S's key has $|R|$ rows. $\{\sigma, \pi, \cup, -, \times, \rho\}$ is a complete set. Division = "for all".

SQL

Order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. Aggregates skip NULL; `COUNT(*)` does not. Comparison with NULL is UNKNOWN; use `IS NULL`. `NOT IN` with a NULL in the list returns nothing. "For all" = double `NOT EXISTS`. Left join keeps unmatched left rows.

Integrity and normalization

Keys: superkey ⊃ candidate (minimal) ⊃ primary. FK may be NULL; PK may not. Closure X^+ decides everything. 2NF: no partial dependency. 3NF: X superkey or A prime. BCNF: X superkey. BCNF⊂3NF⊂2NF⊂1NF. Lossless (binary): common attributes form a key of a piece. BCNF may lose dependencies; 3NF never. Single key of k attributes in n: 2^{n-k} superkeys.

Files and indexes

$bfr = \lfloor B/R \rfloor$, $b = \lceil r/bfr \rceil$. Binary search $\lceil \log_2 b \rceil$. Sparse index: one entry per block (sorted file). Dense: one per record (secondary). B$^+$-tree: data in leaves, order p from $pP+(p-1)K \le B$, height $\approx \log_p N$.

Preprocessing and warehouse

Min–max $\frac{v-\min}{\max-\min}$; z-score $\frac{v-\mu}{\sigma}$; decimal scaling $v/10^j$. Equal-width vs equal-depth bins. Huffman: merge two smallest. Star: denormalised dimensions; snowflake: normalised. Roll-up/drill-down/slice/dice/pivot. Cuboids: 2^n (no hierarchies), $\prod(L_i+1)$ (with). Distributive: sum, count, min, max. Algebraic: avg. Holistic: median.

10 Practice Questions (with answers)

Try each one yourself first. Numeric and SQL answers were checked by running code.

1. An ER diagram has entities E_1, E_2, E_3; E_1–E_2 is M:N, E_2–E_3 is 1:N, E_1–E_3 is 1:1. Minimum number of tables? **Answer:** 4.

2. $R(A, B, C, D)$ has the single candidate key A. Number of superkeys? **Answer:** $2^3 = 8$**.**
3. $R(A, B, C, D, E)$, $F = \{A \to B,\ BC \to E,\ ED \to A\}$. Candidate keys and highest normal form? **Answer:** $ACD,\ BCD,\ CDE$**; 3NF (all attributes prime), not BCNF.**
4. $R(S, C, I)$ with $SC \to I,\ I \to C$. Highest normal form? **Answer: 3NF, not BCNF** ($I \to C$ with I not a superkey).
5. $R(A, B, C)$, $F = \{A \to B\}$. Is $\{AB, BC\}$ a lossless decomposition? **Answer: No** (common attribute B is a key of neither piece).
6. Canonical cover of $\{A \to BC,\ B \to C,\ A \to B,\ AB \to C\}$? **Answer:** $\{A \to B,\ B \to C\}$**.**
7. $R(A, B, C, D)$ with $A \to B,\ C \to D$. Highest normal form? **Answer: 1NF** (key AC; B and D depend on parts of it).
8. $|R| = 1000$, $|S| = 500$. Number of tuples in $R \times S$? **Answer:** 500 000**.**
9. With $R = \{(1, x), (2, x), (3, y), (4, \text{NULL})\}$ and $S = \{(x, 10), (x, 20), (z, 30), (\text{NULL}, 40)\}$: rows in $R \bowtie S$, left outer join, full outer join? **Answer:** 4, 6, 8**.**
10. $E = \{(a, 1), (a, 2), (b, 1), (c, 1), (c, 2), (c, 3)\}$, $C = \{1, 2\}$. What is $E \div C$? **Answer:** $\{a, c\}$**.**
11. On the college database, what does `SELECT COUNT(*) FROM Student WHERE cgpa = NULL` return? **Answer:** 0**.**
12. Same database, grouping by `dept` with `COUNT(*)`, `COUNT(cgpa)` and `AVG(cgpa)`: what is the EE row? **Answer:** $(EE, 2, 1, 7.0)$**.**
13. Names of students with cgpa above the average cgpa? **Answer: Asha and Meena** (average 7.775).
14. `SELECT x FROM A WHERE x NOT IN (SELECT y FROM B)` with $A = \{1, 2, 3\}$, $B = \{1, \text{NULL}\}$. **Answer: empty result.**
15. Number of rows from `SELECT * FROM Student, Course` (5 and 3 rows)? **Answer:** 15**.**
16. Student LEFT JOIN Enroll, `COUNT(e.cid)` per student: what does Tara get? **Answer:** 0 (she appears, with no matches).
17. A referencing-table insert of an Enroll row with a non-existent `sid`: result? **Answer: rejected** (referential integrity violation).
18. 30 000 records of 100 B, block 1024 B: blocking factor and blocks? **Answer:** 10 **and** 3000**.**
19. Worst-case block accesses of binary search on those 3000 blocks? **Answer:** 12**.**
20. Sparse primary index with 14-byte entries on the same file: total block accesses for a key search? **Answer:** 7 (6 for the 42-block index +1).
21. B$^+$-tree internal order for $B = 1024$, key 8, block pointer 6? **Answer:** $p = 73$**.**
22. Min–max value of 21 in $\{4, 8, \ldots, 34\}$ (min 4, max 34)? **Answer:** 0.5667**.**
23. Z-score of 21 (mean 20.33, population $\sigma = 8.966$)? **Answer:** 0.0744**.**
24. Equal-depth smoothing by bin means of $\{4, 8, 9, 15\}$? **Answer: every value becomes** 9**.**
25. Huffman cost of frequencies $45, 13, 12, 16, 9, 5$ versus fixed-length? **Answer:** 224 **vs** 300 **bits.**
26. Number of cuboids in a 3-dimensional cube; with hierarchies $4, 4, 3$ levels-plus-all? **Answer:** 8**;** 48**.**

27. Which of `SUM`, `AVG`, `MEDIAN` are holistic? **Answer: only `MEDIAN`.**

28. In the tiny warehouse, yearly totals after roll-up? **Answer:** 2025: 410**,** 2026: 670**.**

29. Which schema normalises its dimensions: star or snowflake? **Answer: snowflake.**

How to study this section: Spend most of your time on SQL (writing and tracing queries, NULL behaviour, joins, `GROUP BY/HAVING`, subqueries), normalization (closure, candidate keys, normal-form identification, decomposition tests), and the numeric file/index calculations. Then relational algebra and tuple calculus, preprocessing formulas, and warehouse vocabulary (star vs snowflake, OLAP operations, cuboid counts). Hand-solve 10 questions on keys and normal forms and 10 on SQL output, and keep an error log.